

Same Note Can Be Joined With Itself to Duplicate Value

A missing circuit constraint combined with non-atomic nullifier validation enables unbounded value creation and vault drainage

2× value created from one note	1,000 USDT real attacker capital	4,000 USDT withdrawn across two rounds
---	--	---

Target: HackenProof Zendex / zendex-sc-main

Finding: Same-note self-join can mint a spendable note worth double the underlying deposit.

Report created: 25 August 2026

Status at submission: Triaged as valid and sent to the company for review.

Portfolio case study — independent security research

1. Executive Summary

I identified a value-creation vulnerability in `ZendexVaultManager.join()`. The function is intended to combine two distinct notes A and B into a new note C where `amountC` equals `amountA` plus `amountB`. The submitted research found that the system does not enforce that the two input notes are distinct.

Two gaps combine. First, the join circuit does not assert that `nullifierA` and `nullifierB` differ. Second, `ZendexVaultManager.join()` validates both nullifiers as unused before consuming either one. When the same nullifier is supplied twice, both validations therefore pass before either consumption occurs.

The result is that one legitimate deposit can be used as both inputs to `join()`, producing a new, spendable commitment worth double the real deposit. The same operation can be repeated, including compounding an already-doubled note.

The supplied Foundry PoC demonstrates the complete economic consequence: an attacker deposits 1,000 USDT, self-joins the note, withdraws 2,000 USDT, then repeats the technique. In the second round the real vault balance falls by exactly 1,000 USDT beyond the attacker's genuine contribution, with the vault funded by a separate 1,000,000 USDT victim deposit.

2. Finding at a Glance

Target	Zendex / zendex-sc-main
Primary component	<code>ZendexVaultManager.join()</code> + <code>circuits/join/src/main.nr</code>
Root cause	No distinct-input constraint + validate-all-before-consume nullifier handling
Attack requirement	Any legitimately owned note and ability to submit <code>join()</code>
Measured first round	1,000 USDT deposit → 2,000 USDT withdrawal
Measured second round	Another 1,000 USDT contribution → 1,000 USDT net vault loss
PoC	Foundry 1.7.1; 1 test; passing
Submission status	Triaged as valid; sent to company review

3. Technical Root Cause

Gap 1 — the join circuit permits the same note twice.

The submitted report identifies the absence of an explicit distinctness constraint in `circuits/join/src/main.nr`. The circuit verifies each input nullifier and commitment independently, but does not require `nullifierA != nullifierB`, `rhoA != rhoB`, or `commitmentA != commitmentB`.

```
verify_nullifier(nullifier_a, rho_a, pk_x, pk_y); verify_nullifier(nullifier_b, rho_b, pk_x, pk_y);
verify_commitment(commitment_a, asset_id, amount_a, rho_a, pk_x, pk_y); verify_commitment(commitment_b,
asset_id, amount_b, rho_b, pk_x, pk_y); verify_commitment(commitment_c, asset_id, amount_c, rho_c, pk_x,
pk_y);
```

With the same underlying note used for both inputs, `amountA` and `amountB` can both represent the same real deposit, while the join constraint still permits `amountC = amountA + amountB`.

Gap 2 — both nullifiers are checked before either is consumed.

```
_validateNullifier(jp.nullifierA); _validateNullifier(jp.nullifierB); ... _consume(jp.nullifierA);
_consume(jp.nullifierB);
```

Because the first validation does not consume the nullifier, the second validation sees the same nullifier as unused. The subsequent `TreeOperator.consume()` operation is a blind state write, so the second consumption does not invalidate the already-passed validation.

The two defects reinforce one another: the circuit permits the duplicated input, while the contract's ordering fails to reject the duplicated nullifier.

4. Exploit Mechanics

- Attacker owns a legitimate note worth X .
- Attacker supplies the same nullifier as both `nullifierA` and `nullifierB`.
- Two independently generated inclusion proofs can reference the same underlying commitment while using different tags/salts.
- The join circuit accepts both inputs because no distinctness assertion exists.
- `join()` validates both copies before consuming either.
- A new commitment is inserted representing $2X$.
- The attacker withdraws the new note and receives $2X$ of real assets from a vault backed by only X of attacker capital.
- The resulting note can be self-joined again, enabling further multiplication.

5. Proof of Concept

The supplied PoC reports that it compiles and passes against the real, unmodified Zendex contracts. The test deploys the real ZendexVaultManager, TreeOperator, and ZendexVault used by the demonstrated path.

The PoC uses a MockVerifier so the test can run entirely inside Foundry. Importantly, the source explicitly distinguishes this from the root cause: the vulnerability is a missing constraint in the real join circuit, and the report states that a real Noir/Barretenberg prover can generate an on-chain-verifiable proof for the self-join condition.

Forge 1.7.1 build 4072e48	1 test passed	5,901,644 gas
-------------------------------------	-------------------------	-------------------------

Attacker's only real capital: 1,000 USDT After self-join: 2,000 USDT note After self-join + withdraw: 2,000 USDT Net profit created from nothing: 1,000 USDT Vault balance before second round: 999,000 USDT Vault balance after second round: 998,000 USDT Net vault loss in second round: 1,000 USDT

6. Reproduction Sequence

- A victim deposits 1,000,000 USDT into the real vault.
- Attacker deposits 1,000 USDT as their only genuine capital.
- Attacker generates two inclusion proofs referencing the same real commitment, with different tags.
- Attacker calls join() with nullifierA == nullifierB and a new commitment claiming 2,000 USDT.
- The new commitment is inserted into the real tree.
- Attacker withdraws 2,000 USDT.
- Attacker repeats the same technique with another 1,000 USDT deposit.
- The PoC observes the real vault balance decrease by exactly the free 1,000 USDT created in the second round.

7. Economic Impact

This is not merely a duplicate-record or accounting inconsistency. The forged note is spendable against real vault assets.

- **Direct value creation:** one real deposit can produce a note representing twice its backing.
- **Vault insolvency mechanism:** withdrawals of the fabricated value are funded by other users' real deposits.
- **Repeatability:** the technique can be executed repeatedly with fresh notes.
- **Compounding:** an already-doubled note can itself be self-joined again, enabling further multiplication.
- **Minimal transaction complexity:** a single successful join() creates the duplicated note; unlike capacity-exhaustion attacks, the effect does not require hundreds of preparatory transactions.

The supplied PoC pins the second-round loss exactly: the attacker contributes 1,000 USDT and the vault loses 1,000 USDT beyond that contribution. This is stronger evidence than simply observing a changed balance.

8. Remediation

Primary — circuit-level defense: add a distinct-input constraint such as `assert(nullifier_a != nullifier_b)`, or an equivalent rho/commitment inequality, so the two input notes must genuinely differ.

Defense in depth — contract-level ordering: validate and consume each nullifier atomically before validating the next one:

```
_validateNullifier(jp.nullifierA); _consume(jp.nullifierA); _validateNullifier(jp.nullifierB);  
_consume(jp.nullifierB);
```

Under this ordering, supplying the same nullifier twice causes the second validation to observe that it has already been consumed and revert, even if the circuit-side constraint has not yet been deployed.

The submitted report also recommends reviewing every other multi-nullifier operation for the same validate-all-before-consume pattern.

9. Validation Notes

The PoC records that every insertion, consumption, and root lookup goes through the real `TreeOperator`, driven by real `ZendexVaultManager.deposit()`, `join()`, and `withdraw()` calls. The exploit helper supplies two inclusion proofs for the same real commitment and feeds the same nullifier into both join inputs.

The source also notes that different tags are used for the two inclusion proofs. The stated reason is that the relevant verification constraint checks each tag independently against its own inclusion proof; the tags therefore do not prevent both proofs from referencing the same underlying commitment.

10. Research Methodology

- **Invariant analysis:** identify what `join()` is supposed to preserve — two distinct input notes whose values sum to a new note.
- **Circuit-to-contract tracing:** compare constraints enforced in Noir with assumptions made by Solidity.
- **Nullifier lifecycle analysis:** inspect validation and consumption ordering rather than treating nullifiers as automatically atomic.
- **Economic validation:** test whether a fabricated commitment can be converted into real withdrawable assets.
- **End-to-end PoC:** measure both the attacker's balance and the vault's balance to establish where the extracted value comes from.

This finding is a strong example of cross-layer smart-contract security research: the exploitable state emerges from the interaction between a ZK circuit's missing constraint and the Solidity nullifier lifecycle.

11. Triage Status

At the time represented by the submitted report, HackenProof had triaged the finding as valid and sent it to the company for review. The portfolio case study therefore records the finding as a disclosed, triaged vulnerability rather than assigning a later severity or bounty outcome not contained in the supplied materials.

12. Portfolio Notes

This case study represents independent security research and vulnerability disclosure, not commissioned client work. It demonstrates Solidity/DeFi security analysis, ZK-circuit review, nullifier-state reasoning, economic invariant analysis, and Foundry-based exploit validation.

Suggested skills: Smart Contract Security · Solidity · DeFi

Suggested tools: Foundry · Solidity · GitHub

Report date: 25 August 2026

PoC test: `JoinSelfDoubleSpendTest.test_joinNoteWithItself_duplicatesValue_thenDrainsRealVictimFunds()`

Source note: technical claims and measured figures in this case study are based on the submitted vulnerability report and the supplied Foundry PoC. The PoC identifies the exact test result, exploit sequence, scope, and balance assertions.